

CSCE 465/765: Computer & Network Security

Homework Assignment 2: Protect Agent Messages with Classic Cryptography

Mauricio Aguilar Alonso

<https://github.com/MauricioA5/csce465hw2>

Task 0 – Lab Preparation

```
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec$ python3 --version
```

Python 3.12.3

```
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec$ openssl version
```

OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)

```
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec$ pip show cryptography pytest
```

Name: cryptography

Version: 49.0.0

Summary: cryptography is a package which provides cryptographic recipes and primitives to Python developers.

Home-page: <https://github.com/pyca/cryptography>

Author:

Author-email: The Python Cryptographic Authority and individual contributors

<cryptography-dev@python.org>

License-Expression: Apache-2.0 OR BSD-3-Clause

Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages

Requires: cffi

Required-by:

Name: pytest

Version: 9.1.1

Summary: pytest: simple powerful testing with Python

Home-page: <https://docs.pytest.org/en/latest/>

Author: Brianna Laugher, Bruno Oliveira, Floris Bruynooghe, Freya Bruhin, Holger Krekel,

Others (See AUTHORS), Ronny Pfannschmidt

Author-email:

License-Expression: MIT

Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages

Requires: iniconfig, packaging, pluggy, pygments

Required-by:

Task 1 - Demonstrating why encryption alone is insufficient

The program for this task encrypts the fixed command {"action":"READ","path":"notes.txt"} using an AES key and a 16 bit counter. The relay knows the command format, but not the key. The attack uses the fact that the position of READ is known and can be modified with an equal four letter message through an XOR operation. By applying an XOR difference, the ciphertext can be modified to decrypt a different message such as one containing SEND instead of READ.

The replay attack makes use of the attacker sending an encrypted message multiple times. Despite the message being encrypted, the attacker could send the message multiple times and force the relay to process it. During the test, the relay processed the command multiple times even though it was only meant to be processed once. This serves to demonstrate that without authentication, commands can still be modified in transit or even just be used without any modifications, both of which are things encryption alone cannot prevent.

Proof:

```
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec/hw2$ python baseline_ctr.py
AES-CTR Baseline
Original plaintext : b'{"action":"READ","path":"notes.txt"}'
Ciphertext (hex)   : e95a62a70275cf4de62f50e171233443f001612e8a8697f6694980f2ddcfbcc1a22ce582
Decrypted plaintext : b'{"action":"READ","path":"notes.txt"}'

Relay bit-flip attack
Original bytes      : 52454144
Replacement bytes  : 53454e44
XOR difference     : 01000f00
Target offset      : 11
Modified ciphertext: e95a62a70275cf4de62f50e0712c3443f001612e8a8697f6694980f2ddcfbcc1a22ce582
Modified plaintext : b'{"action":"SEND","path":"notes.txt"}'

Replay attack
Sending the same ciphertext twice:
Receiver processed: b'{"action":"READ","path":"notes.txt"}'
Receiver processed: b'{"action":"READ","path":"notes.txt"}'
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec/hw2$
```

Task 2 - Authenticated finite-field Diffie–Hellman setup

The gateway and node create a shared secret using Diffie-Hellman. Both sides generate a private value and exchange their public values. Both sides then calculate the same secret without sending it directly.

DH by itself does not prove who is on the other end. To implement authentication, both sides use RSA signatures. This is done by signing a transcript containing both identities, both DH public values, and two random numbers (nonces). The transcript constraints protocol and group names. Each field has its length written before it so the program can tell exactly where each field starts and ends.

Signatures include the sender's role. This is used to prevent attackers from trying to present gateway signatures as node signatures. Signing the identities, public values, and nonces also means that changing those details makes the signature fail verification. The nonces help distinguish new exchanges from old ones.

After signature verification, both sides use the transcript hash and shared secret to generate encryption and MAC keys. Different labels produce separate keys for each purpose and each direction. As a result, no same key is used for everything.

Fresh DH values also support forward secrecy. If an RSA signing key is stolen later, it does not reveal earlier secrets, assuming old private DH values and session keys were discarded.

Without signatures, an attacker could pretend to be the node when talking to the gateway and pretend to be the gateway when talking to the node. The attacker could then establish separate secrets with both sides and read or change messages passing between them.

The program demonstrates this exchange locally and checks that both sides calculate matching secrets and keys. It generates new RSA keys each time it runs, so it simulates identity keys rather than storing long-term keys between runs.

Proof:

```
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec/csce465hw2$ python3 handshake.py
Authenticated Diffie-Hellman Handshake
Gateway identity : gateway
Node identity    : node
Gateway nonce    : b0e98bfd86a55062d1ab0e5651955427
Node nonce       : 0bf9a07d631f87b7b4326055b7159d9c
Transcript hash  : eb5a638b902939535ff4e345979bc1316e090df85ffd0281f74dacd7da36f
e5d
Gateway verified node signature.
Node verified gateway signature.
DH shared secrets match.
Derived session keys match.
Session ID       : c1d54c5a9ae51892

Handshake successful.
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec/csce465hw2$
```

Task 3: Encrypt-then-MAC record layer

The goal in this task was to combine encryption with a MAC so that the receiver could detect changes to a message. The `seal()` function encrypts the message with AES-CTR, then uses HMAC-SHA-256 to create an authentication tag covering the header, IV, and encrypted message.

The header contains the version, direction, sequence number, message type, and encrypted message length. The IV is built from the session identifier and sequence number. Each direction uses separate encryption and MAC keys.

The `open_record()` function checks the record's format, direction, and expected sequence number. It then checks the authentication tag before decrypting. If any check fails, it raises an error without returning plaintext. This prevents modified messages from being passed to the application.

Sequence numbers start at zero in each direction. After accepting a message, the receiver should expect the next number, so resending old messages should cause it to be rejected. The functions responsible for this take sequence numbers as input, so the main program calling them is responsible for tracking and updating said numbers.

Task 4 - Adversarial Tests

For Task 4, I wrote six tests to check normal communication and several attacks. The first test checks that the handshake succeeds and that messages can be encrypted and decrypted in both directions.

Two tests change a protected message: one changes the ciphertext, and the other changes the message type in the header. Both must be rejected since the authentication tag no longer matches.

The replay test sends an old message after the receiver expects the next sequence number. The receiver rejects it because its sequence number is incorrect. Another test sends a gateway-to-node record to the receiver expecting the opposite direction, which rejects it for having the wrong direction.

The final test checks a handshake signature using an unrelated RSA public key. Verification must fail because that key does not belong to the signer.

Together, these tests show the program accepting valid communication while rejecting the attacks being tested.

Results:

```
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec$ pytest -v hw2/tests
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 -- /home/ubuntu/csce465-agentsec/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/ubuntu/csce465-agentsec
collected 6 items

hw2/tests/test_security.py::test_valid_handshake_and_bidirectional_messages PASSED [ 16%]
hw2/tests/test_security.py::test_modified_ciphertext_rejected PASSED [ 33%]
hw2/tests/test_security.py::test_modified_header_rejected PASSED [ 50%]
hw2/tests/test_security.py::test_replayed_record_rejected PASSED [ 66%]
hw2/tests/test_security.py::test_reflected_record_rejected PASSED [ 83%]
hw2/tests/test_security.py::test_incorrect_rsa_public_key_rejected PASSED [100%]

===== 6 passed in 1.57s =====
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec$
```

Task 4 - Security Note

The attacker in this assignment can observe, change, inject, replay, and redirect messages between the gateway and node. However, the attacker does not know the secret keys and does not break AES, HMAC, Diffie-Hellman, or RSA. It also assumes both endpoints trust the correct RSA public keys. The goal is to protect messages traveling between those endpoints. It does not protect against a compromised endpoint, prevent an attacker from blocking communication, or determine the safety of executing a command.

AES-CTR protects confidentiality by combining plaintext with a keystream using XOR. This makes counter uniqueness essential. If the same counter block is used again with the same encryption key, the same keystream bytes are produced.

Attackers can XOR the corresponding ciphertexts to obtain the XOR of their plaintexts. If one plaintext is known, the attacker can recover the corresponding bytes of the other. As such, avoiding repeated IVs is necessary, counter blocks used throughout different messages must also avoid overlap.

This demonstrates a limitation in IV construction. The IV contains the session identifier followed by the sequence number, while CTR increments its counter for each block within a message. As a result, the second block of a message with sequence zero uses the same counter input as the first block of a message with sequence one. This overlap weakens the confidentiality of consecutive multi block messages.

The receiver verifies the HMAC before decrypting the ciphertext. This is important because CTR alone does not prevent attackers from attempting changes to a message. The HMAC covers the header, IV, and ciphertext, so it protects both the encrypted content and the information used to interpret it. If verification fails, the receiver raises an error without returning plaintext. Checking authentication first keeps attacker-modified plaintext from reaching the application, where it might otherwise be acted upon.

Separate encryption and MAC keys give each key one purpose. The handshake also creates separate keys for gateway-to-node and node-to-gateway traffic.

The different derivation labels distinguish these uses, while the transcript hash connects the keys to the authenticated handshake. Direction checks help reject reflected records, and sequence checks help reject replayed records. The functions in the current implementation do not maintain the sequence themselves. A caller must prevent reuse and advance only after a message is successfully accepted.

An AEAD mode such as AES-GCM combines encryption and authentication through a standard interface. It can authenticate a header as associated data without encrypting it. Compared with manually combining AES-CTR and HMAC, this reduces the number of separate steps that need to be coordinated. However, AES-GCM still requires correct nonce handling, and it does not automatically reject replays of previously valid messages. Replay protection still requires the application or protocol to track freshness.

Finally, valid protected records can still contain dangerous authorized tool calls. Authentication can show that the message was produced by an appropriate key holder and was not modified in transit. It does not show whether the content of the message is reasonable or permitted. For example, authenticated requests could ask tools to delete important files or reveal sensitive information. Compromised endpoints could also generate valid tags for harmful requests. As such, applications must also check permissions, validate commands, and restrict tool usage. Cryptography protects communication, but authorization and safe execution are outside its responsibilities.